Kajetan Frątczak - lab 1-2

Zadanie

- Założenie: macierz A - kwadratowa rozmiar n x n; wektory x, y rozmiar n
- Napisać programy w języku Julia do obliczania: **uogólnionego** iloczynu skalarnego dot(x, **A**, y) wektorów z użyciem LinearAlgebra.dot oraz mnożenia macierzy kwadratowej przez wektor (czyli obliczania A*x) z użyciem operatora *.
- Uruchomić i zmierzyć czasy działania obydwu funkcji - każdej dla różnych rozmiarów wektorów n. Uwaga: rozmiary dobrać tak, żeby nie zużyć zbyt wiele pamięci, a jednocześnie uzyskać niezerowe wyniki czasowe (wielkości będą zależeć od mocy sprzętu - można próbować testować w okolicach n=1000). Dokonać 10 pomiarów dla każdego rozmiaru wektora.
- Czasy działania powinny być zapisywane do **jednego** pliku CSV. Należy zaplanować odpowiednią strukturę kolumn takiego pliku.
- Wczytać dane z w/w pliku do **jednego** DataFrame w języku Julia.
- Korzystając z mechanizmów DataFrame w języku Julia obliczyć średnią i odchylenie standardowe, w taki sposób, aby narysować wykresy średnich czasów obliczenia operacji w zależności od rozmiaru wektora. Dodać słupki błędów do obydwu wykresów uzyskanych z obliczenia odchylenia standardowego.
- Proszę poukładac wykresy w tabelkę za pomocą opcji layouts: https://docs.juliaplots.org/latest/layouts/#Simple-Layouts-1.
- Należy zadbać o staranne podpisanie osi i wykresów.

1. Implementacja programów w Julii

```julia
using CSV, Statistics, DataFrames, Plots, LinearAlgebra
n_values = [750, 1000, 1250, 1500, 1750, 2000, 3000, 4000, 5000]
output = "results.csv"

# DataFrame do przechowywania wyników
results = DataFrame(n = Int[], function_name = String[], time_elapsed = Float64[])

# Pomiary czasu
for n in n_values
    A = rand(n, n)
    # println(typeof(A)) = Matrix{Float64}
    x = rand(n)
    # println(typeof(x)) = Vector{Float64}
    y = rand(n)

    time_for_dot = Float64[]
    time_for_mult = Float64[]

    for i in 1:10
        # dot(x, A, y) = generalized dot product dot(x, A*y) (bardziej przyjazna składnia)
        time_dot = @elapsed dot(x, A, y)
        push!(time_for_dot, time_dot)

        time_multiplication = @elapsed A*x
        push!(time_for_mult, time_multiplication)
    end

    for t in time_for_dot
        push!(results, (n, "dot", t))
    end

    for t in time_for_mult
        push!(results, (n, "multiplication", t))
    end
end

CSV.write(output, results)

data = CSV.read(output, DataFrame)
```

180×3 DataFrame

Row	n	function_name	time_elapsed
	Int64	String15	Float64
1	750	dot	0.0004799
2	750	dot	0.0008262
3	750	dot	0.0007614
4	750	dot	0.0008394
5	750	dot	0.0008598
6	750	dot	0.0010389
7	750	dot	0.0009336
8	750	dot	0.000389
9	750	dot	0.0009454
10	750	dot	0.0009158
11	750	multiplication	0.0041929
12	750	multiplication	0.0039308
13	750	multiplication	0.0024825
⋮	⋮	⋮	⋮
169	5000	dot	0.0327047
170	5000	dot	0.0355539
171	5000	multiplication	0.0197102
172	5000	multiplication	0.0194157
173	5000	multiplication	0.0156631
174	5000	multiplication	0.0155843
175	5000	multiplication	0.0144552
176	5000	multiplication	0.0188105
177	5000	multiplication	0.0157689
178	5000	multiplication	0.0163776
179	5000	multiplication	0.0187018
180	5000	multiplication	0.0167263

```
# Pogrupowałem dane według nazwy funkcji, żeby zobaczyć różnice.
data_grouped=groupby(data, :function_name)
```

GroupedDataFrame with 2 groups based on key: function_name

First Group (90 rows): function_name = "dot"

Row	n	function_name	time_elapsed
	Int64	String15	Float64
1	750	dot	0.0004799
2	750	dot	0.0008262
3	750	dot	0.0007614
4	750	dot	0.0008394
5	750	dot	0.0008598
6	750	dot	0.0010389
7	750	dot	0.0009336
8	750	dot	0.000389
9	750	dot	0.0009454
10	750	dot	0.0009158
11	1000	dot	0.0007895
12	1000	dot	0.0011985
13	1000	dot	0.0007967
⋮	⋮	⋮	⋮
79	4000	dot	0.0149551
80	4000	dot	0.0145329
81	5000	dot	0.0392499
82	5000	dot	0.0280798
83	5000	dot	0.0381966
84	5000	dot	0.0204195
85	5000	dot	0.0227148
86	5000	dot	0.023078
87	5000	dot	0.0311477
88	5000	dot	0.0345785
89	5000	dot	0.0327047
90	5000	dot	0.0355539

Last Group (90 rows): function_name = "multiplication"

Row	n	function_name	time_elapsed
	Int64	String15	Float64
1	750	multiplication	0.0041929
2	750	multiplication	0.0039308
3	750	multiplication	0.0024825
4	750	multiplication	0.001499
5	750	multiplication	0.0018716
6	750	multiplication	0.0015447
7	750	multiplication	0.001987
8	750	multiplication	0.0019851
9	750	multiplication	0.0010876
10	750	multiplication	0.0015049
11	1000	multiplication	0.0009884
12	1000	multiplication	0.0018543
13	1000	multiplication	0.0032128
⋮	⋮	⋮	⋮
79	4000	multiplication	0.0095805
80	4000	multiplication	0.0106527
81	5000	multiplication	0.0197102
82	5000	multiplication	0.0194157
83	5000	multiplication	0.0156631
84	5000	multiplication	0.0155843
85	5000	multiplication	0.0144552
86	5000	multiplication	0.0188105
87	5000	multiplication	0.0157689
88	5000	multiplication	0.0163776
89	5000	multiplication	0.0187018
90	5000	multiplication	0.0167263

```
# Testowe liczenie średniej
mean(data.time_elapsed)
```

```
0.007156112222222223
```

```
# Testowe liczenie odchylenia standardowego
std(data.time_elapsed)
```

```
0.00810107868051499
```

```julia
# Grupowanie danych według funkcji i rozmiaru wektora
data_grouped = groupby(data, [:function_name, :n])

# Obliczenie średniej i odchylenia standardowego dla każdej funkcji i rozmiaru n
grouped = combine(data_grouped, :time_elapsed => mean => :mean_time, :time_elapsed => std => :std_time)

# Podział danych na dwie grupy
dot_data = filter(row -> row.function_name == "dot", grouped)
multiplication_data = filter(row -> row.function_name == "multiplication", grouped)
```

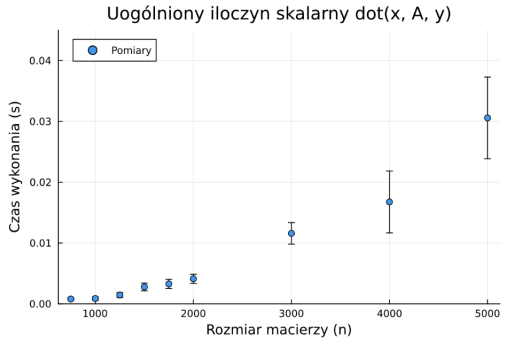
9×4 DataFrame

Row	function_name	n	mean_time	std_time
	String15	Int64	Float64	Float64
1	multiplication	750	0.00220861	0.00104813
2	multiplication	1000	0.00192377	0.000681859
3	multiplication	1250	0.00243184	0.00108664
4	multiplication	1500	0.00263658	0.00094583
5	multiplication	1750	0.00260347	0.00105423
6	multiplication	2000	0.00459543	0.00207773
7	multiplication	3000	0.0103317	0.00235884
8	multiplication	4000	0.0127407	0.00362036
9	multiplication	5000	0.0171214	0.00186943

2. Analiza wyników

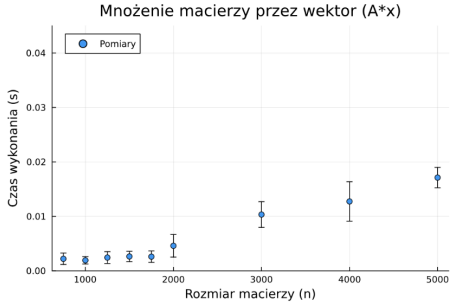
[27]:
```julia
# Wykres 1
plot1 = scatter(dot_data.n, dot_data.mean_time, yerr=dot_data.std_time, seriestype=:scatter,
    label="Pomiary", xlabel="Rozmiar macierzy (n)", ylabel="Czas wykonania (s)", title="Uogólniony iloczyn skalarny dot(x, A, y)", ylims=(0,0.045), legend=:topleft)
```

[27]:



[28]:
```julia
# Wykres 2
plot2 = scatter(multiplication_data.n, multiplication_data.mean_time, yerr=multiplication_data.std_time, seriestype=:scatter,
    label="Pomiary", xlabel="Rozmiar macierzy (n)", ylabel="Czas wykonania (s)", title="Mnożenie macierzy przez wektor (A*x)", ylims=(0,0.045), legend=:topleft)
```

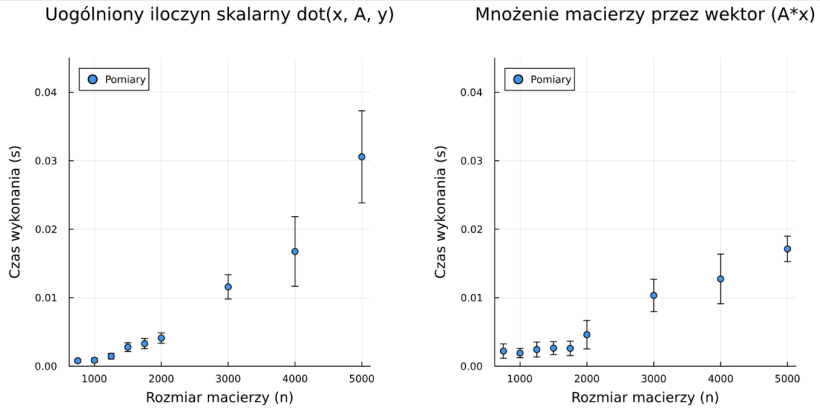
[28]:



[ ]:
```julia
# Małe wyjaśnienie:
# yerr - dodanie słupków błędów
# seriestype = :scatter - określenie rodzaju wykresu (scatter plot)
# label = "dot(x, A, y)" - opis legendy
# xlabel = "Rozmiar n" - podpis osi X
# ylabel = "Średni czas (s)" - podpis osi Y
# title = "Dot product computation" = tytuł wykresu
```

```
[29]: # Układ wykresów w tabelce
plot(plot1, plot2, layout=(1,2), size=(1000, 500), margin=10Plots.mm)
```

[29]:



3. Krótkie podsumowanie

Czas wykonania obu operacji rośnie wraz ze wzrostem rozmiaru wektorów/macierzy, co jest zgodne z teoretyczną złożonością obliczeniową tych operacji. Im większe dane, tym większe różnice w wynikach testów - świadczy to o lekkiej niedokładności operacji na liczbach zmiennoprzecinkowych.